

ISM2411 — Lab Week 06

Sales Loop — Sum, Average, Max — Instructor Facilitation Guide

ISM2411 · Python for Business · USF Muma College of Business

Module 06 · Unit 2 · Control Flow & Structure

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	2
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	5
6.1 Intro (0:00–0:05)	5
6.2 Exercise 1 — Sum a List (0:05–0:15, 10 min)	5
6.3 Exercise 2 — Average (0:15–0:21, 6 min)	7
6.4 Exercise 3 — Max (0:21–0:29, 8 min)	8
6.5 Exercise 4 — Filtered Count (0:29–0:36, 7 min)	10
6.6 Exercise 5 — break / continue (0:36–0:46, 10 min)	11
6.7 Exercise 6 — while Loop Rewrite (0:46–0:54, 8 min)	13
6.8 Exercise 7 — Discount Applied to All Items (0:54–1:02, 8 min)	15
6.9 Stretch 1 — Running Total Report (1:02–1:10, 8 min)	17
6.10 Stretch 2 Preview — Nested Loops (as time allows)	18
7 Wrap-Up (last ~5 minutes)	20
8 Appendix A — Full Answer Key (sales_loop.py)	20
9 Appendix B — Extra Practice (only if the class finishes early)	23

1 Session Snapshot

Course	ISM2411 — Python for Business
Session	Module 06 Lab — Sales Loop: Sum, Average, Max
Unit	Unit 2 · Control Flow & Structure
Class length	75 minutes
Format	Live code-along
Prerequisites	Module 05: if/elif/else; Modules 01–04: variables, operators, f-strings
Student-facing lab page	markumreed.github.io/ism2411 — week06_lab
Exercises covered	Exercises 1–7 (required) + Stretch 1/2 (as time allows)
Submission	sales_loop.py to Canvas

This is the module where Python stops being “one calculation at a time” and starts processing real *data sets* — a whole list of sales, not one transaction. The **accumulator pattern** (initialize, loop, update) introduced in Exercise 1 is the single most important idea in the entire lab; every later exercise is a variation of it. If students leave today able to explain, unprompted, why an accumulator needs to be initialized *before* the loop starts, this lab succeeded — everything else (max-tracking, filtered counting, break/continue) is the same skeleton with small variations.

2 Learning Objectives

By the end of this 75-minute session, students should be able to:

1. Write the accumulator pattern from scratch: initialize a variable before the loop, update it inside the loop, use the final value after the loop.
2. Explain why `total` must be initialized to `0` *before* the loop, not inside it — and why doing it wrong either crashes or silently resets on every iteration.
3. Combine an accumulator with an `if` check inside the loop body to filter or conditionally update.
4. Use `break` to exit a loop early and `continue` to skip an iteration, and articulate the difference between the two.
5. Convert a `for` loop into an equivalent `while` loop, and explain the tradeoffs of each.

3 Before Class — Setup Checklist

- ☐ Open `sales_loop.py`, empty except the header comment.

- ❑ Have the exact sales list ready to type/paste identically every time: `sales = [120, 80, 250, 175, 90, 410, 60, 215]` — using the *same* list across all seven exercises means students build intuition for what each new technique does differently on data they already know the shape of.
- ❑ Decide in advance how much of Exercise 6 (the `while` loop rewrite) to live-code vs. narrate — it’s intentionally the most tedious exercise in the lab (that tedium *is* the lesson), so don’t feel obligated to type every line slowly if the room is already following the accumulator pattern well by that point.

4 Materials Needed

- Instructor laptop + terminal + editor, Python 3.10+
- Students: `sales_loop.py`, same project structure as prior modules

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:05	Welcome: “processing a whole list, not one value”	5
0:05–0:15	Exercise 1 — Sum a list (the accumulator pattern)	10
0:15–0:21	Exercise 2 — Average	6
0:21–0:29	Exercise 3 — Max	8
0:29–0:36	Exercise 4 — Filtered count	7
0:36–0:46	Exercise 5 — <code>break</code> / <code>continue</code>	10
0:46–0:54	Exercise 6 — <code>while</code> loop rewrite	8
0:54–1:02	Exercise 7 — Discount applied to all items	8
1:02–1:10	Stretch 1 — Running total report	8
1:10–1:15	Stretch 2 preview + wrap-up, reflection, submission checklist	5

Seven required exercises plus Stretch 1 fill the full 75 minutes; Stretch 2 (nested loops / multiplication table) is a genuinely separate topic (loops *inside* loops) rather than a variation on today’s

accumulator pattern, so it's positioned as a preview/take-home unless the room finishes unusually early.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:05)

Say to the class:

“Every script so far has worked with one value at a time — one cart total, one transaction. Today we process a whole list: eight sales figures, and we compute a sum, an average, and a max — all with the exact same underlying pattern, called the accumulator pattern. Learn that pattern well today and the rest of this lab is just small variations on it.”

Do: Open `sales_loop.py`, type the header and the shared list every exercise will reuse:

```
# sales_loop.py
# ISM2411 Module 06 Lab – Sales Loop: Sum, Average, Max
sales = [120, 80, 250, 175, 90, 410, 60, 215]
```

6.2 Exercise 1 — Sum a List (0:05–0:15, 10 min)

Teaching goal: The accumulator pattern, in its simplest form — this is the exercise to slow all the way down for, since it’s the load-bearing idea of the whole lab.

Say to the class:

“Three steps, always in this order: initialize a variable before the loop starts, update it on every pass through the loop, use the final value after the loop ends. I want you to say those three words back to me — initialize, update, use — because you’re going to apply this exact skeleton six more times today.”

Live-code this:

```
# --- Exercise 1 ---
total = 0                # initialize
for sale in sales:
    total += sale         # update
print(f"Total: ${total}") # use
```

Line-by-line explanation:

- `total = 0` — **initialize**, before the loop. This has to happen *before* `for sale in sales:` starts, not inside it — say explicitly why: if this line were inside the loop body, `total` would be reset to 0 on every single pass, and you’d only ever end up with the *last* sale’s value, not a running sum. This is the single most important sentence in this lab — consider writing it on the board verbatim.

- `for sale in sales:` — this is a for loop over a list. `sale` is a **loop variable**: on the first pass it holds `sales[0]` (120), on the second pass `sales[1]` (80), and so on, automatically, one element at a time, until the list is exhausted. Point out explicitly: `sale` is not a special keyword — it's a name *you* chose, and it could be called anything (`x`, `s`, `amount`); the convention of naming it something meaningful (singular of the list name) is a readability habit, not a rule.
- `total += sale` — **update**, inside the loop, indented to show it's part of the loop body. `+=` means “add `sale` to whatever `total` already holds, and store the result back in `total`.” This line runs eight times, once per element, each time adding that pass's `sale` value onto the running `total`.
- `print(f"Total: ${total}")` — **use**, after the loop, *not indented* — say this explicitly: this line's indentation (back to the left margin, same level as `total = 0`) is what tells Python it runs *once*, after the loop finishes, not once per iteration. A student who accidentally indents this line to match the loop body will get eight separate `Total: ...` print statements, showing the running total after each addition instead of the true final sum — a good deliberate demo if you have a spare 30 seconds.

Run it. Expected output:

Total: \$1400

Common student mistakes to watch for:

- Initializing `total` *inside* the loop (`for sale in sales: total = 0; total += sale`) — resets to 0 every pass, so the final value is just the last element, 215, not 1400. Demonstrate this live; it's the exercise's central teaching point and worth seeing fail before it's understood as correct.
- Forgetting `+=` and writing `total = sale` instead — same failure mode as above, from a different typo; the fix (`total += sale`, or fully spelled out, `total = total + sale`) is worth writing both ways once so students see `+=` is shorthand, not new syntax.
- Indenting the final `print()` to match the loop body, discussed above.

Check for understanding: “If `sales` had 50 items instead of 8, what — if anything — would need to change about this code?” (Nothing — the loop automatically runs once per item regardless of list length; this is worth stating explicitly as the payoff of using a loop at all, versus eight individual hardcoded additions.)

6.3 Exercise 2 — Average (0:15–0:21, 6 min)

Teaching goal: Track a *second* accumulator (a count) alongside the first, and combine them after the loop — the same skeleton, doubled.

Say to the class:

“Average needs two numbers: the total, and how many items there were. We’re tracking both inside the same loop, then dividing after the loop ends — not using `len()` or `sum()`, on purpose, so the accumulator pattern is doing all the work.”

Live-code this:

```
# --- Exercise 2 ---
total = 0      # initialize
count = 0      # initialize
for sale in sales:
    total += sale  # update
    count += 1     # update
average = total / count  # use
print(f"Total: ${total}")
print(f"Count: {count}")
print(f"Average: ${average:.2f}")
```

Line-by-line explanation:

- `count = 0` — a second accumulator, initialized alongside `total`, before the loop.
- `count += 1` — inside the loop, incrementing by exactly 1 on every pass, regardless of the sale’s value — contrast this explicitly with `total += sale`, which adds a *different* amount each time. Say plainly: `count` is counting iterations; `total` is summing values. Two different jobs, same pattern.
- `average = total / count` — this line runs **after** the loop, using both accumulators’ final values together. It could not run correctly *inside* the loop, since `count` and `total` aren’t at their final values until the loop finishes — flag this explicitly, since some students will be tempted to compute a “running average” inside the loop out of habit.

Run it. Expected output:

```
Total: $1400
Count: 8
Average: $175.00
```

Common student mistakes to watch for:

- Computing average inside the loop body — produces eight different intermediate “averages,” none of which is the real answer; ask a student to explain why the *last* one printed still wouldn’t

be correct even though it's computed last (because it's `total / count` at whatever partial values they held on that specific pass, not the true final total divided by the true final count — actually on the very last pass this *does* coincidentally equal the right answer, which is worth pointing out as a subtle trap: it “looks like it works” if you only check the last printed line, hiding the fact that the other seven lines were wrong).

- Dividing by `len(sales)` instead of `count`, defeating the purpose of the exercise (which explicitly asks students to track a count manually) — not wrong output-wise, but redirect to the stated constraint, since Exercise 6 depends on students being comfortable manually tracking values like this without built-ins.

Check for understanding: “Why do we need a *separate* count variable — why can't we just use `total` for both the sum and the count?” (Because they're fundamentally different quantities that happen to both start at 0 — `total` needs the actual sale amounts added in, `count` needs a flat 1 added in regardless of amount; conflating them would corrupt both.)

6.4 Exercise 3 — Max (0:21–0:29, 8 min)

Teaching goal: A third accumulator variant — instead of adding on every pass, *conditionally replace* the tracked value only when a new one is larger.

Say to the class:

“Same three-step skeleton, but the update step is different: instead of always adding, we only update when we find something bigger than what we've seen so far.”

Live-code this:

```
# --- Exercise 3 ---
current_max = 0          # initialize
for sale in sales:
    if sale > current_max: # compare
        current_max = sale # update, conditionally
print(f"Largest sale: ${current_max}")
```

Line-by-line explanation:

- `current_max = 0` — initialized to 0 here specifically because all sales in this list are positive, so 0 is guaranteed to be smaller than any real sale and will always get replaced on the first comparison. Flag this as a real limitation worth naming: **if the list could contain negative numbers, initializing to 0 would be wrong** — a list of all-negative sales would incorrectly report a max of 0, which was never actually in the data. The safe, general-purpose alternative is to initialize `current_max` to the list's *first element* (`sales[0]`) instead of a hardcoded 0 — mention this explicitly as the more robust pattern, even though 0 works for this specific

dataset.

- `if sale > current_max:` — the comparison step, using Module 05's `if` syntax, now nested inside a loop for the first time this semester. This is worth calling out as a structural first: a conditional *inside* a loop, which is exactly the combination Exercise 4 and 7 will lean on even harder.
- `current_max = sale` — the conditional update: only runs on passes where the `if` above was `True`. Walk through the first three sales by hand on the board: $120 > 0 \rightarrow$ update to 120; $80 > 120 \rightarrow$ False, no update; $250 > 120 \rightarrow$ update to 250. This concrete trace is worth doing out loud.

Run it. Expected output:

Largest sale: \$410

Common student mistakes to watch for:

- Updating `current_max` unconditionally every pass (forgetting the `if`, or misindenting so the assignment isn't actually inside the `if` block) — this just sets `current_max` to the *last* element in the list every time, silently giving the wrong answer whenever the last element isn't actually the largest. With this specific list (last element 215, true max 410), this bug is very visible if you test it live.
- Using `>=` instead of `>` in the comparison — harmless for this exercise (doesn't change the final answer), but worth a quick “why doesn't this matter here, and when might it?” aside: it would only matter if you were also tracking *which* sale index was the max and cared about ties.

Check for understanding: “Using the same skeleton, how would you track the *minimum* instead of the maximum — what two things change?” (Initialize to a very large number, or to `sales[0]`, instead of `0`; flip the comparison to `<`. Getting a student to articulate both changes confirms they understand the pattern's moving parts, not just this one instance of it.)

6.5 Exercise 4 — Filtered Count (0:29–0:36, 7 min)

Teaching goal: Combine *two* accumulators with a filtering `if`, directly previewing Exercise 7’s structure — this is the exercise that makes the “if inside a loop, with an accumulator” combination feel routine rather than novel.

Say to the class:

“Now we’re counting and summing, but only for sales that pass a filter — over \$100. Notice this uses everything from the last three exercises at once.”

Live-code this:

```
# --- Exercise 4 ---
count_over_100 = 0      # initialize
total_over_100 = 0      # initialize
for sale in sales:
    if sale > 100:       # filter
        count_over_100 += 1    # update
        total_over_100 += sale # update
print(f"Sales over $100: {count_over_100}")
print(f"Total of those sales: ${total_over_100}")
```

Line-by-line explanation:

- Two accumulators initialized before the loop, same as Exercise 2 — but this time, **both updates are indented one level deeper**, inside the `if`, not directly inside the `for`. This double-indentation is worth pointing at explicitly on screen: the `if sale > 100:` line is inside the loop body (one level of indent), and both `+=` lines are inside the `if` body (two levels of indent) — get this indentation wrong and the filter either does nothing or excludes everything.
- Both accumulators update together, only for sales that pass the filter — sales at or below \$100 are simply skipped for both counters, since neither line inside the `if` body runs when the condition is `False`.

Run it. Expected output:

```
Sales over $100: 5
Total of those sales: $1170
```

Common student mistakes to watch for:

- Indenting `total_over_100 += sale` back out to the loop level instead of the `if` level — this makes it sum *every* sale regardless of the filter, silently disconnecting it from `count_over_100`, which correctly stays filtered. A good “the two numbers don’t seem to agree” bug for the room to catch by comparing their own total against the full-list total from Exercise 1 (\$1170 vs. \$1400 — if a student gets \$1400 here, that’s the tell).

- Using `>=` vs `>` for the \$100 threshold — worth a quick check against the lab’s stated boundary (over \$100, meaning strictly greater), and a reminder that boundary wording (“over,” “at least,” “more than”) should be read carefully every time, since Module 05 already established these distinctions matter.

Check for understanding: “If I wanted the *average* of just the sales over \$100, what one extra line would I add, and where?” (`average_over_100 = total_over_100 / count_over_100`, after the loop ends — reusing Exercise 2’s pattern on this exercise’s filtered accumulators, confirming the skeletons genuinely compose.)

6.6 Exercise 5 — `break` / `continue` (0:36–0:46, 10 min)

Teaching goal: Two new loop-control keywords that change a loop’s normal top-to-bottom, every-iteration behavior — `continue` skips the rest of the *current* pass, `break` exits the *entire* loop immediately.

Say to the class:

“So far every loop has run through the whole list, every time. These two keywords change that. `continue` says ‘skip the rest of this one pass, move to the next item.’ `break` says ‘stop the loop completely, right now, don’t look at any more items.’ Very different, easy to confuse — let’s build intuition by using both in the same loop.”

Live-code this:

```
# --- Exercise 5 ---
for sale in sales:
    if sale <= 100:
        continue                # skip small sales
    if sale > 400:
        print(f"Alert: outlier found – ${sale}. Stopping.")
        break                    # stop entirely
    print(f"${sale}")
```

Line-by-line explanation:

- `if sale <= 100: continue` — for any sale of 100 or less, `continue` immediately jumps to the next iteration of the loop, skipping every line below it for this pass — specifically, the `print()` at the bottom never runs for these values. Say explicitly: `continue` does not stop the loop, it only stops the *current pass through* the loop.
- `if sale > 400: ... break` — for any sale over \$400, print the alert message, then `break` — this exits the loop **immediately and completely**, and no further sales in the list are examined at all, even if some of them would also be over \$100 or under \$400. This is the crucial contrast

with `continue`: after a `break`, the loop is entirely finished; there's no "next iteration" to move on to.

- `print(f"${sale}")` — the fallback case: only reached by sales that are *both* over \$100 (didn't trigger `continue`) *and* not over \$400 (didn't trigger `break`).

Trace it by hand with the class, sale by sale, since this is the exercise's real payoff: 120 → passes both checks, prints \$120. 80 → `continue`, skipped, nothing printed. 250 → prints \$250. 175 → prints \$175. 90 → `continue`, skipped. 410 → triggers `break`, prints the alert, **loop ends here** — the remaining sales (60, 215) are never even looked at.

Run it. Expected output:

\$120

\$250

\$175

Alert: outlier found — \$410. Stopping.

Common student mistakes to watch for:

- Expecting the loop to continue after `break`, printing 60 and 215 too — this is the single most common misconception; explicitly confirm with the room that the output has exactly four lines, not more, and ask someone to explain out loud why 60 and 215 never appear.
- Confusing which keyword does which — a genuinely common mix-up; if a student swaps them, walk them through re-tracing by hand rather than just naming the correct keyword, since the tracing is what builds real understanding.
- Placing the `continue` check *after* the `break` check — with this specific data, swapping the order changes nothing about the final output (since 410 fails the `<= 100` check either way and would reach the `break` check either way) but it's worth asking the room whether order between these two particular checks matters here, and why (it doesn't, because the two conditions — `sale <= 100` and `sale > 400` — can never both be true for the same value, so which one is checked "first" doesn't change the outcome; contrast this explicitly with Module 05 Exercise 1's discount tiers, where order *did* matter, to keep reinforcing when order is and isn't significant).

Check for understanding: "If I removed the `break` entirely, keeping only the `continue`, what would the output be for the full list?" (\$120, \$250, \$175, \$410, \$215 — every sale over \$100 gets printed now, including \$410 and \$215, since nothing stops the loop early anymore; only the `<= 100` sales are still skipped.)

6.7 Exercise 6 — while Loop Rewrite (0:46–0:54, 8 min)

Teaching goal: Re-implement Exercise 1’s sum using a while loop and a manually managed index — direct, felt comparison of the two loop styles, including the tedium of manual index management that for loops hide.

Say to the class:

“Same sum as Exercise 1, same final answer — but built with a while loop instead of a for loop. You’re going to feel the difference immediately: while doesn’t know how to walk through a list on its own. You have to manage the position yourself, with an index variable.”

Live-code this:

```
# --- Exercise 6 ---
total = 0          # initialize
i = 0             # initialize the index
while i < len(sales):
    total += sales[i] # update, using the index to look up the value
    i += 1           # update the index – THIS LINE IS ESSENTIAL
print(f"Total: ${total}")
```

Line-by-line explanation:

- `i = 0` — a second thing to initialize now: not just the accumulator, but also an **index**, a running position counter starting at the list’s first valid position, 0.
- `while i < len(sales):` — the loop continues running as long as this condition stays True. `len(sales)` is 8; the loop runs for `i = 0, 1, 2, ..., 7` and stops the moment `i` reaches 8, since `8 < 8` is False.
- `total += sales[i]` — unlike the for loop’s `sale` variable (which directly *held* each value), here we have to manually **look up** the value at position `i` using `sales[i]` — this is genuinely more work, and worth naming as such.
- `i += 1` — **this line is the one students most often forget**, and forgetting it causes an infinite loop: if `i` never increases, `i < len(sales)` stays True forever, and the program hangs. If this happens live, that’s a valuable (if slightly nerve-wracking) demo — show how to interrupt it (Ctrl+C in the terminal), then point at the missing line as the cause.

Run it. Expected output (identical to Exercise 1):

Total: \$1400

Have students add a one-line comment answering the exercise’s actual required question: which version (this while loop, or Exercise 1’s for loop) would they use in production, and why. **Model a strong answer:** “The for loop is shorter, and it’s impossible to forget to increment — there’s no separate index to manage. I’d use while only when I don’t actually know in advance how many times I need to loop (e.g., ‘keep asking the user for input until they type quit’), which isn’t the

situation here.”

Common student mistakes to watch for:

- Forgetting `i += 1`, causing an infinite loop, discussed above.
- Off-by-one errors from an incorrect condition (e.g., `i <= len(sales)` instead of `i < len(sales)`) — this attempts to access `sales[8]`, which doesn’t exist (valid indices are 0 through 7), raising `IndexError: list index out of range`. Good moment to connect back to Module 01’s “reading an unfamiliar error” skill — have a student read the error message and identify what it’s telling them, rather than you translating it for them.

Check for understanding: “What are the *minimum* two things a `while` loop needs that a `for sale in sales:` loop gets for free?” (An explicitly initialized index variable, and an explicit increment step inside the loop body — a `for` loop over a list handles both automatically.)

6.8 Exercise 7 — Discount Applied to All Items (0:54–1:02, 8 min)

Teaching goal: Reuse Module 05's tiered-discount if/elif/else logic *inside* a loop, applying it independently to every element in a list — the exercise that most directly bridges last week's conditionals with this week's loops.

Say to the class:

"Last module's tiered discount logic, but instead of running it once for one cart total, we run it once per sale in this list — eight independent discount decisions, one per pass through the loop."

Live-code this:

```
# --- Exercise 7 ---
for sale in sales:
    if sale >= 200:
        discount = 0.10
    elif sale >= 100:
        discount = 0.05
    else:
        discount = 0
    discounted_price = sale * (1 - discount)
    print(f"${sale} → {discount*100:.0f}% discount → ${discounted_price:.2f}")
```

Line-by-line explanation:

- The entire if/elif/else block is now **inside** the for loop, indented one level — meaning it runs fresh, from scratch, on every single pass, independently deciding a new discount value for each sale. Point out explicitly: unlike the accumulators in Exercises 1–4, discount is *not* carried over between iterations — it's recomputed every time, with no memory of the previous sale's discount.
- `discounted_price = sale * (1 - discount)` — Module 04/05's discount formula, applied per-item inside the loop.
- The `print(...)` line runs on every pass too, giving one line of output per sale — contrast this explicitly with Exercises 1–4, where `print()` ran *once*, after the loop, using an accumulated final value. This is a genuinely important structural distinction worth naming: **does this `print()` belong inside the loop (one line per item) or after it (one summary line)?** — both are valid patterns, and choosing correctly is a real design decision every future loop-based script will require.

Run it. Expected output (first two of eight lines):

```
$120 → 5% discount → $114.00
$80 → 0% discount → $80.00
```

(Full output has eight lines, one per sale in the original list, in order.)

Common student mistakes to watch for:

- Writing the `if/elif/else` block *outside* the loop by mistake (unindented) — this computes a `discount` once, using whatever `sale` happened to hold after the loop finished (its *last* value, 215), and then applies that same single discount to nothing further, since the loop already ended — a `SyntaxError`-free but structurally broken script, worth tracing through carefully if it happens.
- Reusing a `discount` variable name that collides with an accumulator from an earlier exercise still in the same file — a good moment to reinforce that once Exercise 6 or 7 begins, this file should really be thought of as several logically separate scripts sharing one file, and variable names from one exercise’s block can silently leak into the next if you’re not careful about what’s already defined.

Check for understanding: “How many separate `if/elif/else` *decisions* does this script make in total when it runs?” (Eight — one fresh decision per sale, not one decision applied to all of them. Confirming this count is a good check that the “runs fresh every iteration” idea actually landed.)

6.9 Stretch 1 — Running Total Report (1:02–1:10, 8 min)

Teaching goal: A bank-statement-style running total — print the accumulator’s value *during* the loop, at every step, not just at the end — plus a genuinely interesting discussion point about what “count how many times it crossed \$500” even means for strictly increasing data.

Say to the class:

“One more accumulator variation: print the running total after every single transaction, like a bank statement, instead of only printing the final sum once at the end.”

Live-code this:

```
# --- Stretch 1 ---
running_total = 0
crossed_500 = False
crossings = 0

for sale in sales:
    running_total += sale
    print(f"${sale} -> running total: ${running_total}")
    if running_total > 500 and not crossed_500:
        crossings += 1
        crossed_500 = True

print(f"Number of transactions that first pushed the running total above $500: {crossings}")
```

Line-by-line explanation:

- `running_total += sale` then immediately `print(...)` — **inside the loop**, so the printed value updates on every single pass, unlike Exercise 1 where the total was only revealed once, at the very end.
- `crossed_500 = False` — a **flag** variable: a boolean that starts `False` and gets set to `True` the first time (and only the first time) the running total exceeds \$500. This is a new accumulator *shape* — not a running sum or count of *values*, but a one-time “has this happened yet” tracker.
- `if running_total > 500 and not crossed_500:` — combines a numeric check with the flag, using `and not` (Module 05’s Exercise 6 pattern) to make sure this only fires once — without the `not crossed_500` guard, `crossings` would increment on *every remaining pass* once the total first crosses \$500, since `running_total > 500` stays `True` for every sale after that point (sales are all positive, so the running total only ever goes up).
- **Worth raising as an explicit discussion point:** since every sale in this list is positive, `running_total` is *monotonically increasing* — it only ever goes up, never down. That means it can only cross the \$500 threshold **exactly once**, no matter what the data looks like, which is why `crossings` will always report 1 for this dataset. Ask the room: “under what circum-

stances could this count be something other than 1?” (If the list contained negative values — refunds or returns — the running total could dip back below \$500 after crossing it, then cross again later, making a genuine multi-crossing count meaningful. With today’s all-positive sales data, the flag guard is still good practice, but the “count” is a bit of a red herring on this particular dataset — a good moment for honest, real critique of an exercise’s design, which is itself a useful habit to model.)

Run it. Expected output (final line only shown; full output has one line per sale):

\$120 -> running total: \$120

\$80 -> running total: \$200

\$250 -> running total: \$450

\$175 -> running total: \$625

\$90 -> running total: \$715

\$410 -> running total: \$1125

\$60 -> running total: \$1185

\$215 -> running total: \$1400

Number of transactions that first pushed the running total above \$500: 1

Common student mistakes to watch for:

- Omitting the `crossed_500` flag entirely and just counting every pass where `running_total > 500` — this overcounts badly (five of the eight passes have a running total over \$500, so crossings would incorrectly report 5 instead of 1), a good concrete illustration of why the flag is necessary.

Check for understanding: “Which specific sale caused the running total to first cross \$500?” Have the room trace it together rather than eyeballing it: \$120→\$120, \$80→\$200, \$250→\$450, \$175→\$625 — it’s the **fourth** sale, \$175, that pushes the running total over \$500, from \$450 to \$625. This is worth genuinely walking through together, since it’s an easy one to get wrong by guessing instead of tracing.

6.10 Stretch 2 Preview — Nested Loops (as time allows)

Frame as a quick preview if time is short:

“One loop inside another — a nested loop. Classic first example: a multiplication table. For every row i from 1 to 5, and for every column j from 1 to 5, print $i \times j$. That’s 25 total multiplications from just two short loops.”

If time allows a live demo:

```
for i in range(1, 6):
    row = ""
    for j in range(1, 6):
```

```
row += f"{i*j:4}"  
print(row)
```

State the business-relevant extension verbally even if you don't code it live: "swap the multiplication table for 5 products $\times$ 3 quantity tiers, and you have a full price list generated by the exact same nested-loop shape — the answer key has the working version if you want to explore it before next class."

7 Wrap-Up (last ~5 minutes)

Review the reflection questions out loud:

1. *Explain the accumulator pattern as if to a classmate who missed lecture.* — A strong answer names all three steps explicitly (initialize before, update inside, use after) and explains *why* each matters, not just what each does — specifically, why initializing before the loop and using after the loop are both required for correctness, not just style.
2. *while loop vs for loop code length and tradeoffs* — expect students to note the `while` version needed an explicit index and increment, and is more error-prone (infinite loops, off-by-one) — but is also more flexible for situations where you don't know the loop count in advance.
3. *A real dataset from their major, with a loop-based summary statistic* — push for specificity: what's in the list, what gets accumulated, what gets printed at the end. “I'd use a loop for my finance internship” is not a complete answer; “a list of daily stock closing prices, accumulating a running total to compute a monthly average” is.

Review the submission checklist together:

- ☐ File is named `sales_loop.py`
- ☐ Contains Exercises 1–7, each clearly separated
- ☐ Uses the accumulator pattern correctly (initialize before, update inside, use after) throughout
- ☐ `break/continue` used correctly in Exercise 5
- ☐ Script runs top to bottom with no errors

Preview Module 07: “Every exercise today repeated a very similar block of code — declare some variables, loop, accumulate. Next module introduces functions, which let you package that logic up once and reuse it by name, instead of retyping it every time you need it.”

8 Appendix A — Full Answer Key (`sales_loop.py`)

```
# sales_loop.py
# ISM2411 Module 06 Lab – Sales Loop: Sum, Average, Max
sales = [120, 80, 250, 175, 90, 410, 60, 215]

# --- Exercise 1 ---
total = 0
for sale in sales:
    total += sale
print(f"Total: ${total}")

# --- Exercise 2 ---
total = 0
count = 0
```

```

for sale in sales:
    total += sale
    count += 1
average = total / count
print(f"Total: ${total}")
print(f"Count: {count}")
print(f"Average: ${average:.2f}")

# --- Exercise 3 ---
current_max = 0
for sale in sales:
    if sale > current_max:
        current_max = sale
print(f"Largest sale: ${current_max}")

# --- Exercise 4 ---
count_over_100 = 0
total_over_100 = 0
for sale in sales:
    if sale > 100:
        count_over_100 += 1
        total_over_100 += sale
print(f"Sales over $100: {count_over_100}")
print(f"Total of those sales: ${total_over_100}")

# --- Exercise 5 ---
for sale in sales:
    if sale <= 100:
        continue
    if sale > 400:
        print(f"Alert: outlier found - ${sale}. Stopping.")
        break
    print(f"${sale}")

# --- Exercise 6 ---
total = 0
i = 0
while i < len(sales):
    total += sales[i]
    i += 1
print(f"Total: ${total}")

```

```

# Would use the for loop in production: shorter, and there's no
# separate index to forget to increment.

# --- Exercise 7 ---
for sale in sales:
    if sale >= 200:
        discount = 0.10
    elif sale >= 100:
        discount = 0.05
    else:
        discount = 0
    discounted_price = sale * (1 - discount)
    print(f"${sale} → {discount*100:.0f}% discount → ${discounted_price:.2f}")

```

Stretch 1 (Running total report):

```

running_total = 0
crossed_500 = False
crossings = 0

for sale in sales:
    running_total += sale
    print(f"${sale} → running total: ${running_total}")
    if running_total > 500 and not crossed_500:
        crossings += 1
        crossed_500 = True

print(f"Number of transactions that first pushed the running total above $500: {crossings}")

```

Stretch 2 (Nested loops — multiplication table, then price list):

```

for i in range(1, 6):
    row = ""
    for j in range(1, 6):
        row += f"{i*j:4}"
    print(row)

products = ["Widget", "Gadget", "Gizmo", "Doohickey", "Thingamajig"]
tiers = [1, 5, 10]
price_per_unit = 25
for product in products:
    for qty in tiers:

```

```
if qty >= 10:
    discount = 0.10
elif qty >= 5:
    discount = 0.05
else:
    discount = 0
total_price = price_per_unit * qty * (1 - discount)
print(f"{product} x{qty}: ${total_price:.2f}")
```

9 Appendix B — Extra Practice (only if the class finishes early)

Seven required exercises plus Stretch 1 fill the full 75 minutes at a normal pace. If a section moves unusually fast:

Extra — a second accumulator pass, different data. `expenses = [45, 120, 15, 300, 60, 90]`. Have students independently compute, using the accumulator pattern from scratch (no peeking at Exercise 1–4’s code): total, count, average, and max, without looking at their earlier code. (Total: \$630. Count: 6. Average: \$105.00. Max: \$300.)

Extra — a second break/continue scenario. Using `expenses` above, print each expense that’s under \$100, but stop entirely the moment an expense of \$300 or more is found. Have students trace it by hand before running: 45 is under 100, prints. 120 is neither under 100 nor at/over 300 — it matches neither condition, so nothing happens for it at all, and the loop just moves on with no output for that item. 15 prints. 300 triggers the stop. Expected printed lines: \$45, \$15, then a stopping message — a good extra rep specifically because, unlike Exercise 5’s version, the “skip” and “stop” conditions here are not mutually exclusive by construction, and working out that 120 triggers neither is a genuinely useful exercise in careful condition-reading.